

```

FROM
  "projectauthorizations"
WHERE
  "projectauthorizations"."userid" = 4156052
  AND (projectauthorizations.projectid =
projectfeatures.projectid)
  AND (projectauthorizations.accesslevel >= 10))))
AND "issues"."stateid" = 1
AND ("issues"."projectid" IS NULL
  OR "projects"."archived" = FALSE)
AND "issues"."workitemtypeid" IN (1, 2, 5, 7, 6, 3)
ORDER BY
  "issues"."createdat" DESC, "issues"."id" DESC
LIMIT 21

```

Such queries can easily take 3-10 seconds to run on the current production database. A similar query that counts the total number of issues for the gitlab-org group currently times out. Although these queries are fairly well supported by database indexes, the primary issue lies in the large volume of data that PostgreSQL needs to scan something that can't be further optimized.

Optimizations have been attempted in the past to improve the performance of such queries, but essentially, these optimizations won't solve all the scaling issues associated with the hierarchical database queries:

- Addition of namespaces.traversalids to our namespaces table as an optimization for fast hierarchy lookup.
- IN-operator optimization
- Hierarchy caching

The problem gets more complicated if we start extending our features to support organizational level lookups. Organizations can contain several group hierarchies which multiplies the data the database will need to scan.

Fundamentally, we have reached the limit of how far we can optimize hierarchical queries. This is making, and will continue to make, delivering results for some of our largest customers very difficult. Engineers see this today when developing features and it becomes very difficult to write performant queries for very large group/project hierarchies.

Overview of the optimization in DIP

Within the Data Insights Platform, optimizing this query would involve a few key steps. These steps are fairly general, so the same approach could be applied to other group-level features like merge requests or epics.

Data synchronization

Siphon (blueprint) enables low-latency replication of PostgreSQL tables to other database

systems, typically with just 1.5 seconds of data lag. This is low enough to treat the data as consistent and usable for "real-time" queries.

The replicated tables maintain the same structure as they have in PostgreSQL, making integration straightforward.

projectauthorizations table in PostgreSQL:

sql

```
CREATE TABLE projectauthorizations (  
  userid bigint NOT NULL,  
  projectid bigint NOT NULL,  
  accesslevel integer NOT NULL,  
);
```

ALTER TABLE ONLY projectauthorizations

ADD CONSTRAINT projectauthorizationspkey PRIMARY KEY (userid, projectid, accesslevel);

siphonprojectauthorizations table in ClickHouse:

sql

```
CREATE TABLE gitlabclickhousedevdevelopment.siphonprojectauthorizations (  
  userid Int64,  
  projectid Int64,  
  accesslevel Int64,  
  siphonreplicatedat DateTime64(6, 'UTC') DEFAULT now(),  
  siphondeleted Bool DEFAULT false  
)
```

-- define the version and delete columns

ENGINE = ReplacingMergeTree(siphonreplicatedat, siphondeleted)

PRIMARY KEY (userid, projectid)

ORDER BY (userid, projectid)

Database table engine for Siphon

Siphon tables in ClickHouse (by convention, table names start with the siphon prefix) use a specialized table engine called ReplacingMergeTree, which is optimized for handling data changes.

ClickHouse doesn't support efficient single-row updates or deletes · there's no UPDATE statement in the traditional sense. Instead, changes are handled through inserts, and the engine deduplicates data in the background based on the primary key(s).

Deduplication is controlled by two special columns managed by Siphon:

- siphonreplicatedat: the version of the row, usually a timestamp
- siphondeleted: a flag indicating whether the row has been deleted

An "update" is just an INSERT with a newer version value:

```
sql
INSERT INTO siphonprojectauthorizations
(userid, projectid, accesslevel, siphonreplicatedat, siphondeleted)
VALUES (1, 1, 10, '2025-04-06 18:12:52.234564', false);
```

A "delete" is also an INSERT, but with siphondeleted = true and a newer version value:

```
sql
INSERT INTO siphonprojectauthorizations
(userid, projectid, accesslevel, siphonreplicatedat, siphondeleted)
VALUES (2, 1, 10, '2024-04-06 13:43:05.786542', true);
```

At any given time, multiple rows with the same primary key might exist.

Example table data:

userid	projectid	accesslevel	siphonreplicatedat	siphondeleted
1	1	30	2025-04-05 03:10:43.118265	false
2	1	10	2024-01-03 02:32:37.703416	false
1	2	10	2025-03-14 09:07:22.847632	false
1	1	10	2025-04-06 18:12:52.234564	false
2	1	10	2024-04-06 13:43:05.786542	true

Although the table contains five rows, deduplication should result in just one row. Deduplication is automatic, asynchronous and handled by ClickHouse:

```
sql
SELECT FROM siphonprojectauthorizations FINAL
```

userid	projectid	accesslevel	siphonreplicatedat	siphondeleted
1	1	10	2025-04-06 18:12:52.234564	false

Note: FINAL is not recommended for production use as it might collapse/merge rows in query time.

How did we get here?

Let's look at the primary key pairs:

- (1, 1): The first row is overridden by the fourth row, which has a newer version and lower access level.
- (2, 1): The second row is overridden by the fifth row, which marks it as deleted.

You can simulate the effect of deduplication at query time with the following:

```
sql
SELECT FROM (
  SELECT
    userid,
    projectid,
    argMax(accesslevel, siphonreplicatedat) AS accesslevel,
    argMax(siphondeleted, siphonreplicatedat) AS deleted
  FROM siphonprojectauthorizations
  GROUP BY userid, projectid
) pa
WHERE deleted = false
```

Note: This deduplication pattern is essential when working with ReplacingMergeTree tables to avoid returning two records with the same primary key.

Query performance for Siphon tables

Siphon database tables are optimized for accessing data via the defined primary key(s). The primary key definition for the tables are always exactly the same as the PostgreSQL counterpart.

This means that queries targeting other columns will not perform well. This is an example for filtering issues for a project in ClickHouse:

```
sql
SELECT
FROM siphonissues
WHERE
  projectid = 278964
ORDER BY createdat, id
LIMIT 20;
```

Note: for simplification, the deduplication part is not added to the example queries.

Performance is very slow, the query scans the whole table.

```
plaintext
20 rows in set. Elapsed: 40.830 sec. Processed 100+ million rows.
```

The group-level query would likely perform worse, as we would need to query all projects or namespaces for the group hierarchy from the siphonnamespaces table.

Optimizing queries for a different access pattern other than the primary key in ClickHouse is not as easy as simply adding a database index. Database indexes are effective if the primary key is also filtered so the data to be scanned is already reduced.

Materialized Views on top of Siphon tables

Materialized views in ClickHouse let us define efficient access patterns for our database tables. Think of them as INSERT triggers; whenever data is inserted into the source table, the materialized view automatically gets updated too.

This works well for us because both UPDATE and DELETE operations in ClickHouse are represented as INSERTs. As long as the primary key doesn't change (see the caveats section), the materialized view will stay in sync with the source table.

Here's an example of a materialized view that makes it faster to query by projectid:

```
sql
CREATE TABLE issuesbyproject
(
  -- list the same columns as in siphonissues
)
ENGINE = ReplacingMergeTree(siphonreplicatedat, siphondeleted)
PRIMARY KEY (projectid, id)
ORDER BY (projectid, id);

CREATE MATERIALIZED VIEW issuesbyprojectmv TO issuesbyproject
AS SELECT
FROM siphonissues;
```

Running queries against this new materialized view performs much better:

```
sql
SELECT
FROM issuesbyproject
WHERE
  projectid = 278964
ORDER BY createdat, id
LIMIT 20;
```

```
plaintext
20 rows in set. Elapsed: 0.024 sec. Processed 125.83 thousand rows
```

As you can see, materialized views help optimize queries with specific filters. Even though

ClickHouse processed over 125,000 rows, that's still considered efficient · unlike in PostgreSQL, scanning 1.5 million rows per query is pretty normal in ClickHouse due to its low I/O cost.

Adding more filters to the query won't affect the performance significantly, as long as those filters are targeting other columns in the table. For example, adding a filter on authorid would perform just as well as the query above.

You can also improve performance further with optimizations like data skipping indexes, if needed.

Materialized view for hierarchical lookups

The previous example was addressing projectid based lookups which can be optimized quite well in PostgreSQL with a database index. Going back to the original problem; How can we optimize data access queries in the group hierarchy?

The key element for the optimization is the traversalids array from the namespaces table which contains all ancestors for a given namespace record.

A namespace record can be:

- Top-level group
- Subgroup
- Project (called: ProjectNamespace)
- Personal namespace (called UserNamespace)

This is the traversal ids value of the Siphon project on GitLab.com: [9970,55154808,95754906]

- 9970: gitlab-org top-level group.
- 55154808: analytics-section sub-group.
- 95754906: siphon project namespace.

The project namespace has a 1:1 mapping with the projects table. In this case it points to a project row with id=63106760.

Organization, group, subgroup and project level queries can be optimized by making a materialized view where all issues are stored with the respective traversalids value.

Formatting of the traversalids

ClickHouse is not very efficient when filtering dynamic array prefixes. Turning traversalids into a string yields much better performance. We can also add the organization into the mix, by simply prepending the organization id.

The traversalids becomes traversalpath: 1/9970/55154808/95754906/ (1 is the organization id)

Building on top of the siphonnamespaces table, we can create a materialized view to create the formatted traversalpath:

```

sql
CREATE TABLE namespacetraversalpaths(
  id Int64 DEFAULT 0,
  traversalpath String DEFAULT '0/',
  version DateTime64(6, 'UTC') DEFAULT now(),
  deleted Bool DEFAULT false
)
ENGINE = ReplacingMergeTree(version, deleted)
PRIMARY KEY id
ORDER BY id
SETTINGS indexgranularity = 512; -- optimize single-row value lookups

CREATE MATERIALIZED VIEW namespacetraversalpathsmv TO namespacetraversalpaths
(
  id Int64,
  traversalpath String,
  version DateTime64(6, 'UTC'),
  deleted Bool
)
AS SELECT
  id,
  -- concat organizationid and the traversalids array
  if(length(traversalids) = 0, concat(toString(ifNull(organizationid, 0)), '/'),
  concat(toString(ifNull(organizationid, 0)), '/', arrayStringConcat(traversalids, '/'), '/') AS
  traversalpath,
  siphonreplicatedat AS version,
  siphondeleted AS deleted
FROM gitlabclickhousedevdevelopment.siphonnamespaces

```

With this materialized view we can easily query the traversalpath value for a given namespace id:

```

sql
SELECT traversalpath
FROM namespacetraversalpaths
WHERE
id = 95754906
LIMIT 1;

-- 1/9970/55154808/95754906/

```

Creating the issues materialized view

Creating the materialized view involves a JOIN where we connect the issues records via the namespaceid column with the namespacetraversalpaths table.

The materialized view will contain all columns from the siphonissues table plus one extra

column called traversalpath from the JOIN above. Optionally, columns which doesn't need to be filtered at all can be skipped to save disk space.

sql

```
CREATE TABLE IF NOT EXISTS hierarchyissues
```

```
(
    traversalpath String,
    id Int64,
    title Nullable(String),
    authorid Nullable(Int64),
    projectid Nullable(Int64),
    createdat Nullable(DateTime64(6, 'UTC')),
    updatedat Nullable(DateTime64(6, 'UTC')),
    milestoneid Nullable(Int64),
    iid Nullable(Int64),
    updatedbyid Nullable(Int64),
    weight Nullable(Int64),
    confidential Bool DEFAULT false,
    duedate Nullable(Date32),
    movedtoid Nullable(Int64),
    timeestimate Nullable(Int64) DEFAULT 0,
    relativeposition Nullable(Int64),
    servicedeskreplyto Nullable(String),
    lasteditedat Nullable(DateTime64(6, 'UTC')),
    lasteditedbyid Nullable(Int64),
    closedat Nullable(DateTime64(6, 'UTC')),
    closedbyid Nullable(Int64),
    stateid Int8 DEFAULT 1,
    duplicatedtoid Nullable(Int64),
    promotedtoepicid Nullable(Int64),
    healthstatus Nullable(Int8),
    sprintid Nullable(Int64),
    blockingissuescount Int64 DEFAULT 0,
    upvotescount Int64 DEFAULT 0,
    workitemtypeid Nullable(Int64),
    namespaceid Nullable(Int64),
    startdate Nullable(Date32),
    version DateTime64(6, 'UTC') DEFAULT now(),
    deleted Bool DEFAULT FALSE

```

```
)
```

```
ENGINE = ReplacingMergeTree(version, deleted)
```

```
PRIMARY KEY (traversalpath, id);
```

```
CREATE MATERIALIZED VIEW hierarchyissuesmv TO hierarchyissues
```

```
AS WITH
```

```
cte AS
```

```
(
```

```
    SELECT
```

```
    FROM siphonissues
```

```
),
```

```

namespacepaths AS
(
  -- look up traversalpath values
  SELECT
    id,
    traversalpath
  FROM namespacetraversalpaths
  WHERE id IN (
    SELECT DISTINCT namespaceid
    FROM cte
  )
)
SELECT
  -- handle the case where namespaceid is null
  multilf(cte.namespaceid != 0, namespacepaths.traversalpath, '0/') AS namespacepath,
  cte.id Int64,
  cte.title,
  cte.authorid,
  cte.projectid,
  cte.createdat,
  cte.updatedat,
  cte.milestoneid,
  cte.iid,
  cte.updatedbyid,
  cte.weight,
  cte.confidential,
  cte.duedate,
  cte.movedtoid,
  cte.timeestimate,
  cte.relativeposition,
  cte.servicedeskreplyto,
  cte.lasteditedat,
  cte.lasteditedbyid,
  cte.closedat,
  cte.closedbyid,
  cte.stateid,
  cte.duplicatedtoid,
  cte.promotedtoepicid,
  cte.healthstatus,
  cte.sprintid,
  cte.blockingissuescount,
  cte.upvotescount,
  cte.workitemtypeid,
  cte.namespaceid,
  cte.startdate,
  cte.siphonreplicatedat AS version,
  cte.siphondeleted AS deleted
FROM cte
LEFT JOIN namespacepaths ON namespacepaths.id = cte.namespaceid

```

Evaluating the performance of the materialized view

With the traversalpath column present in the table, we can now run group-level queries efficiently:

```
sql
SELECT
FROM hierarchyissues
WHERE
  startsWith(traversalpath, '1/9970/')
ORDER BY createdat, id
LIMIT 20;
```

```
plaintext
20 rows in set. Elapsed: 0.047 sec. Processed 780.83 thousand rows
```

The query took 47 milliseconds and it scanned only 780000 rows.

Let's look out a COUNT query with state distribution:

```
sql
SELECT stateid, COUNT()
FROM hierarchyissues
WHERE
  startsWith(traversalpath, '1/9970/')
GROUP BY stateid
```

```
plaintext
2 rows in set. Elapsed: 0.007 sec. Processed 780.83 thousand rows
```

We can filter issues down to a particular user by filtering the authorid column:

```
sql
SELECT
FROM hierarchyissues
WHERE
  startsWith(traversalpath, '1/9970/') AND
  authorid = 4156052
ORDER BY createdat, id
LIMIT 20;
```

```
plaintext
20 rows in set. Elapsed: 0.034 sec. Processed 780.83 thousand rows
```

Here an optimization could be done via data skipping indexes:

sql

```
ALTER TABLE hierarchyissues ADD INDEX idxauthorid authorid TYPE set(1000) GRANULARITY 1;
```

This helps ClickHouse reduce the number of processed rows:

plaintext

20 rows in set. Elapsed: 0.022 sec. Processed 421.14 thousand rows

More complex cases

Querying issues often involves several tables and JOINS. With the optimized materialized view, we've significantly reduced the amount of records the database needs to scan. Adding extra filters as JOIN need to be carefully considered.

Let's assume that we also want to filter out issues which is not visible to the current user. This can be done by replicating the projectauthorizations table to ClickHouse. The modified query looks like this:

sql

```
SELECT
FROM hierarchyissues
WHERE
  startsWith(traversalpath, '1/9970/') AND
  projectid IN (
    SELECT projectid FROM siphonprojectauthorizations WHERE userid = 4156052
  )
ORDER BY createdat, id
LIMIT 20;
```

plaintext

20 rows in set. Elapsed: 0.050 sec. Processed 895.52 thousand rows

Note: A similar filter can be added for the banned users. Alternatively, a ClickHouse dictionary could be built for improving the filtering speed.

Direct associations on issues with filters needs to be carefully planned. JOIN statements on the primary key is not as efficient in ClickHouse as in PostgreSQL so queries filtering issue assignees, reviewers or labels will likely need a different solution.

One idea is moving these tables into hierarchyissues where the has-many association becomes an array:

- label links becomes labelids[].

- assignees becomes assigneeids[].
- reviewers becomes reviewerids[].

This means that when one of these associations change, the data in the hierarchyissues table may go out of date. Possible mitigations:

- If the association on the Rails side calls touch on the associated issue (bumping the updatedat timestamp) then Siphon will re-insert the issues row automatically. This means that associations defined in the materialized view will be also automatically reloaded.
- Periodical eventually consistency lookup checks will correct the underlying data. See the caveats section.

Implementation

The Data Insights Platform would ensure that the necessary data is available and up to date in ClickHouse (using Siphon). In case more data is needed on ClickHouse, Siphon would also replicate those tables.

1. Check the current ClickHouse database and Siphon configuration to determine which tables are already replicated.
1. If the table is missing, create the database table in ClickHouse and configure the replication with Siphon.
1. Design a materialized view where the primary key is the traversalpath + the primary key of the source table.
1. Implement the query mechanism by choosing one of the options below.

Initially, there will be two options for querying the data:

- Proto API between DIP and the Rails application.
 - In this case the DIP query API would expose a simple interface, data deduplication and other complexities are hidden.
- Use ClickHouse directly from the Rails application (this approach will be deprecated at some point).
 - The ClickHouse query is implemented in the Rails application by the feature owner.

Additional maintenance tasks:

Siphon would eventually support database schema changes. This means the schema changes between the PostgreSQL tables and Siphon tables will be automatically handled by the platform.

Earlier in the document, it was mentioned that some inconsistencies might happen as the data changes in the materialized view. The DIP platform would ensure that the configured materialized views regularly checked for inconsistencies and data is corrected as quickly as possible.

Conclusion

ClickHouse offers significantly faster query execution times on large volumes of data compared to PostgreSQL. However, the database schema must be carefully designed to align with the application's data access patterns. By leveraging materialized views optimized for hierarchical

lookups (such as organization, group, and project levels) we can enable real-time analytical capabilities. This approach is generic enough to be easily extended to other core domain objects, such as:

- Merge requests
- Epics
- Vulnerabilities
- Other features which are available on the project or group level.

Overview of the components within DIP:

Caveats and Limitations

This proposal presents a relatively complex solution to a scalability problem for which there is currently no other viable alternative.

ClickHouse Query Complexity

Because of the ReplacingMergeTree engine, every query on Siphon tables or materialized views based on them must ensure query-time deduplication. This typically requires additional GROUP BY subqueries, which can make queries harder to generate and more difficult to debug.

Increased Development Costs

The proposal introduces a different database system: ClickHouse. To maintain backward compatibility, queries and filters would need to be maintained with double effort: once for PostgreSQL (existing) and once for ClickHouse (new).

Note: aligning with the proposal in the advanced finders document would somewhat mitigate the costs.

More Moving Parts

The Data Insights Platform introduces more infrastructure components, which also means more potential points of failure:

- Siphon: Handles data synchronization
- ClickHouse: Serves as the analytical data store
- DIP Query API: Acts as the query layer

Data Lag

As with any replication-based system, some level of data lag is inevitable and should be expected. Whilst we can reasonably describe this system as "real-time", it shouldn't be used for time-sensitive operations.

Data Inconsistency

As noted in the materialized views section, most data changes are correctly reflected in the corresponding materialized view tables.

However, there's one edge case where inconsistency can occur. In the hierarchy issue materialized view, if the traversalid column changes, for example when a subgroup or project is moved in the hierarchy, the value will not be automatically updated for existing records.

There is a technique to detect and fix this issue, described in this Siphon issue. Using a periodic background worker, the data lag caused by such hierarchy changes can be significantly reduced.

More information about this problem can be found in this blog post.

SECTION: engineering/architecture/design-
documents/data_insights_platform_querying_api/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

The Data Insights Platform (DIP) is a unified abstraction to ingest, process, persist & query analytical data streams generated across GitLab, enabling our ability to compute business insights across the product.

A part of this work, that needs further definition, is being able to query the analytical data once the data is processed and persisted in the DIP. The querying API must:

- Be easy for customers to use and moves the complexity of interacting with different data tables away from the API, and into the backend service.
- Be consistent for customers to easily identify what data is available, what properties that data has, and how they can interact with it.

This is important because of the current confusion with GitLab APIs in general. The problem we face is that many of the GitLab GraphQL and REST APIs are inconsistent, contributing to the confusion our customers face when interacting with GitLab, and we must avoid compounding that further.

There will be two aspects of the overall querying architecture. The first part, the customers communication with the DIP through our existing GraphQL and REST APIs. The second part, the GitLab rails application's (the Monolith) communication with the DIP through the Data Insights Service.

This proposal references the first part, but is focused on implementing the second.

Goals

- The querying API must support the ClickHouse exporter
 - Although we have plans to support alternative exporters, our core initial goal is to work in conjunction with Siphon to collate and analyse GitLab data, which will be stored in ClickHouse tables.
- Consistent API interface
 - We've had issues in the past with our APIs not being consistent in the type of data being available, and filtering options. This has led to a more confusing user experience.
 - Difficult to accomplish, but we must develop an API which allows different data types and filtering options, without dramatically changing how the API request should be made or handled.
 - Provide support and clear documented instructions on how other GitLab teams can integrate their GraphQL and REST API endpoints that leverage ClickHouse into the DIP querying API.
- Keep complexity away from the end-user APIs
 - The DIP will be handling a wide range of data types and data sources, all being converged into one platform for fully formed analyses. We don't want this complexity to be passed onto the end-user APIs.
 - We need to keep the API straightforward to use; with clear documentation. The user should be informed about the data available to them. They should not be required to know which database or table to use.
- Authentication and authorization
 - The data being collected by the DIP will contain a mix of data privacy categories. We need to ensure that any data being queried is only accessible to those with the correct authentication and authorization. Data outside the purview of the requestor must in no way be accessible to them.
 - Data classification will be determined by the Data Catalog in Atlan.

Non-Goals

- Support for other data sources, such as Snowflake, for this initial phase.
- Integrating all other GraphQL and REST API calls that leverage ClickHouse that aren't owned by Monitor/Platform Insights is out of scope. This work is best executed by the team with domain expertise and it is not scalable for Monitor/Platform Insights to do so for every team. Support and documentation will be provided to enable teams to integrate their endpoints into the DIP.

Historical context

Existing APIs

We have three APIs that were already developed to work with analytical data within their own ClickHouse databases. The APIs were developed independently, and therefore are not compatible with each other in terms of data available, how we interact with the APIs, or how the data is formatted.

The observability and cube API's are not planned to be integral to the querying API, but rather are here for historical context and learning from our experiences with them.

Observability API

Written in Go, the Observability API is REST-based and is used to retrieve data from the GitLab Observability Backend (GOB). The data is stored in a separate ClickHouse database and formatted to work with the OTel format.

The API uses Cloud Connector to authenticate requests and is restricted to singular project requests; rather than being able to span multiple projects within a group.

Users can read and write data directly to GOB through the API's various endpoints:

- Read:
 - get api/v4/projects/[PROJECTID]/observability/v1/analytics
 - get api/v4/projects/[PROJECTID]/observability/v1/traces
 - get api/v4/projects/[PROJECTID]/observability/v1/services
 - get api/v4/projects/[PROJECTID]/observability/v1/metrics
 - get api/v4/projects/[PROJECTID]/observability/v1/logs
- Write:
 - post api/v4/projects/[PROJECTID]/observability/v1/traces
 - post api/v4/projects/[PROJECTID]/observability/v1/metrics
 - post api/v4/projects/[PROJECTID]/observability/v1/logs

This API is not sufficiently documented and requires reading the code within GOB or GitLab to understand what data to expect, and what parameters can be used. However, this is expected as O11y has been withdrawn as a customer offering whilst we work on the DIP.

Cube API

Product Analytics uses Cube to act as a query layer for communicating with the Analytics Stack which uses ClickHouse under-the-hood to store analytics data.

Requests are sent to the Cube proxy which is used to validate the request using Rails authentication, prevent direct API requests to Cube from the frontend, handle any errors, and also do any data transformations needed. The only data transformation currently done is to fill empty data points within a given date range. This transformation is something that can be done using ClickHouse directly, but this isn't supported by Cube and the proposal to add support was rejected by the Cube core team.

Cube uses JSON Web Tokens (JWTs) to define security context within a multi-tenant environment, which is another reason why we are using the proxy. We use Rails authentication to determine if the requestor has permission to get analytics data for the project, then build the JWT with the correct project permissions, which then allows the requestor to retrieve the data from Cube.

From the Cube proxy, Cube will use the pre-defined schemas to generate ClickHouse queries and return the data. Cube provides a consistent query language for users to be able to define queries and return it in a consistent format, with a lot of pre-built filters and formatted data types.

The Cube API is a read-only REST API with the following endpoints:

- get v1/meta
 - Used to generate the list of possible fields, data types, and filtering options for the Data Explorer in conjunction with the Cube data source.
- post v1/load
 - Used to retrieve data for the query provided, the core endpoint for data retrieval.

The API is documented within our REST API documentation. This will need deprecating or removing when we decide to move onto alternative approaches other than Cube.

We have had issues with Cube supporting more advanced features without using their Cube store (we had concerns around storing RED data in their solution). It also doesn't support more advanced features in ClickHouse, such as certain functions or pre-aggregations. As such, there has been a lot of discussion around moving away from Cube for some time.

Optimize

Optimize uses ClickHouse to help keep their aggregated queries for the Contributions, Value Stream, and AI Impact dashboards performant.

For Contributions, data from Postgres is aggregated and added to ClickHouse every 3 minutes. Upon data retrieval, a combination of ClickHouse data and specific Postgres data is used to populate the Contributions dashboard.

Value Stream and AI Impact only use ClickHouse for their data. AI Impact updates its data every 5 minutes rather than the 3 minutes for Contributions and Value stream.

All three dashboards use GraphQL for the frontend to query the Monolith, and the GraphQL resolvers directly call the ClickHouse database using the clickhouseclient gem package maintained by the Optimize team.

The following GraphQL types are used:

- GroupValueStreamAnalyticsFlowMetrics
- ProjectValueStreamAnalyticsFlowMetrics
- GroupValueStreamDashboardUsageOverview
- ProjectValueStreamDashboardUsageOverview
- GroupValueStreams
- ProjectValueStreams
- GroupContributions
- GroupAiMetrics
- ProjectAiMetrics
- GroupAiUserMetrics
- ProjectAiUserMetrics
- aiUsageData (group)
- aiUsageData (project)

GLQL

The GitLab Query Language (GLQL) is an initiative started within the Plan stage to develop a single query language for our customers to interact with their GitLab data.

GLQL is currently built around being used within Markdown blocks, using YAML as its base language. It only supports issues at the time of writing and is specifically developed for use within:

- Wikis (group and project)
- Epics and epic comments
- Issue and issue comments
- Merge requests and merge request comments
- Work items (tasks, OKRs, epics with the new look) and work item comments

The first goal of GLQL is to allow customers to generate auto-updating lists of issues to help them with collaboration and planning. In the medium-term, the Plan stage plans to add support for listing merge requests, epics, and other work items.

Longer-term, GLQL could be the final piece in the data platform puzzle, providing a consistent query format for our customers to interact with all GitLab data, not just Plan-specific data. This would include data from the data platform. However, internal resource limitations, mean that this is a long-term ideal rather than something that could be actioned in FY26.

GLQL is built using Rust. The code is compiled into a WASM frontend module. The module parses the GLQL query before generating the resulting GraphQL query as its output. The generated GraphQL query is then used by the frontend as it usually would to request the resulting data.

Querying API preface

We need to remove the complexity of how the DIP works from the API consumer. Therefore, we must make a querying API which provides a consistent interface for the API consumer which ignores where the data came from, only that this data exists.

Discarded solutions

A few potential solutions were considered before being discarded as unworkable, these include:

- Reusing the existing API implementations
 - Existing implementations rely on external dependencies, which goes against our requirements for the DIP.
 - Product Analytics and Observability will be removed as an offering, therefore the existing implementations are not going to be used.
- Integrating directly with GLQL
 - Written in Rust (reasoning for Rust is documented), which has limited knowledge within the Monitor stage, and would require additional steps to add to the DIP which is primarily Go-based.
 - The GLQL roadmap is Plan stage focused, integrating directly would detract from that roadmap.

Of course, we can use the existing APIs as a learning opportunity for developing the new API.

Chosen solution

We will use the existing GraphQL and REST API infrastructure for UI/customer communication,

with a Protobuf over gRPC API for communication between the Monolith and the DIP.

Later, we will augment these APIs with GLQL integration. GLQL has been built with integrating with existing APIs in mind, whereby the end-user uses GLQL to build their query, and GLQL transforms the request into the underlying API call, so we can go ahead and build our own API, and integrate into the existing GLQL infrastructure later.

We also wouldn't need to worry about delaying Plan's existing roadmap, as we can work with them to find a time to integrate later when GLQL is in GA.

API Structure

We have two existing GitLab API's in GraphQL and REST that will need to communicate with the Monolith's DIP service through rails models.

We will need to create a new Protobuf-over-gRPC API for the Monolith's DIP service to communicate with the DIP itself, and translate the response into something that the Monolith will understand.

The GraphQL and REST API's within the Monolith will not be designed by the team working on the DIP. These API's will be owned by the teams that own the feature. The team working on the DIP will only own and support the integration with the DIP itself.

GraphQL format

The GraphQL API will use the existing GitLab GraphQL service. Therefore, we should follow all the relevant guidelines to developing a new GraphQL endpoint within the service.

The GraphQL API's structure will be determined by the team building the feature. The DIP will not dictate how this API is structured. It is up to the team building the feature to transform the GraphQL request to something that the DIP will understand, and handle the response from the DIP to be returned to the requestor.

We must document how we expect the GraphQL resolvers interact with the DIP service, in terms of expected request and response.

REST format

The REST API will use the existing GitLab REST API. Therefore, we should follow all the relevant guidelines to developing a new REST endpoint within the existing REST API framework.

The REST API's structure will be determined by the team building the feature. The DIP will not dictate how this API is structured. It is up to the team building the feature to transform the REST request to something that the DIP will understand, and handle the response from the DIP to be returned to the requestor.

We must document how we expect the REST handlers interact with the DIP service, in terms of expected request and response.

Due to the nature of the queries we expect to receive, with extensive filtering options such as

filtering by specific projects or users, as well as the potential for queries to contain PII data, the REST endpoints must use POST to send queries to the REST endpoint. Using POST will help us avoid URL length limits (roughly 2,000 characters), and prevent PII data being stored in GET query logs.

POST queries must have size limits applied to the parameters given. This is to prevent users from attempting excessive data requests, such as all MRs from 100K users.

Protobuf format

The Protobuf API must be written using the binary format. Using ProtoJSON would work, but is far less performant, and uses a lot more resources.

The API must follow Protobuf style standards and best practices. It must also make sure that all requests do not exceed the limits outlined by Protobuf. If a response is expected to exceed these limits, it must be made into a stream and sent to the requestor in chunks.

For any API changes, .proto files must be generated for both Go and Rails. These must then be added to the DIP and the Monolith respectively.

Versioning

We must follow the versioning guidelines for GitLab's GraphQL and REST API.

For gRPC, we must version the packages within Go. This will version the gRPC call itself, and allow us to manage version upgrades. For best practice suggestions, Microsoft has a good guide focused on .NET, but is applicable for other languages.

For flagging features within gRPC we must follow the same standard as Gitaly but prefix flags with datainsights instead.

Protobuf is designed to handle versioning automatically by following their best practices. We must not re-use field numbers or field names, any fields removed in future versions must be reserved.

Pagination

For performance reasons, any non-aggregate queries must follow Keyset-based pagination. This approach exposes a link (or cursor) to the next page rather than using page IDs. There should be a cursor for the next and previous pages (when applicable), using column data to generate the appropriate cursor. We do this already for GraphQL.

Aggregate queries are any queries that look to review a small sub-set of data points over a period of time, rather than listing individual rows within the database. In other words, the difference between showing a visualization of merged MRs over time, rather than listing merged MRs in a table.

Aggregate queries must be limited by a restricting factor like a timestamp, but due to their nature can't be paginated. These queries must also be tagged or labelled to note that these are aggregations.

ClickHouse doesn't leverage "traditional" primary keys which means we need to rely on the sorting option to define the next group of items in the query when generating the pagination link. For example, if a user sorts by descending the last updated timestamp, the generated pagination link would filter the next query to be from the previous queries oldest last updated timestamp. In the event that a query doesn't define a sorting option, we must set a default for each endpoint.

Automated schema generation

To keep the API schema up-to-date as efficiently as possible, we should look at automating the schema generation and documentation for the Protobuf API. This could be done on creation of a new version, deployment, or even after every merge. This should then update the relevant documentation.

Example queries

These queries are for example purposes only. They do not reflect what might actually be written, nor should they limit what we end up building during implementation.

Group Contributions

GraphQL query

graphql

```
query getContributionsData($fullPath: ID!, $startDate: ISO8601Date!, $endDate: ISO8601Date!,
$nextPageCursor: String = "", $first: Int) {
  group(fullPath: $fullPath) {
    id
    contributions(
      from: $startDate
      to: $endDate
      first: $first
      after: $nextPageCursor
    ) {
      nodes {
        repoPushed
        mergeRequestsCreated
        mergeRequestsMerged
        mergeRequestsClosed
        mergeRequestsApproved
        issuesCreated
        issuesClosed
        totalEvents
        user {
          id
          name
          webUrl
        }
      }
    }
  }
  pageInfo {
```

```

        endCursor
        hasNextPage
    }
}
}
}

```

GraphQL variables

```

json
"variables": {
  "endDate": "2025-02-13",
  "fullPath": "gitlab-org",
  "nextPageCursor": "",
  "startDate": "2025-02-06"
}

```

Protobuf definition

```

proto
// Editions version of proto3 file

edition = "2023";

package gitlab.group.contributions.v1;

option gopackage = "gitlab.com/api/group/contributions/v1";

import "google/protobuf/timestamp.proto";

service GroupContributionsService {
  // GetGroupContributions retrieves contribution data for a group within a date range
  rpc GetGroupContributions(GetGroupContributionsRequest) returns
  (GetGroupContributionsResponse) {}
}

// Request message for GetGroupContributions
message GetGroupContributionsRequest {
  // Full path of the group (e.g., "gitlab-org")
  string fullpath = 1;

  // Start date for contributions in ISO8601 format (YYYY-MM-DD)
  string startdate = 2;

  // End date for contributions in ISO8601 format (YYYY-MM-DD)
  string enddate = 3;

  // Pagination: next page continuation token from previous response

```

```

optional string nexttoken = 4;

// Pagination: previous page continuation token from previous response
optional string previoustoken = 5;

// Pagination: number of results to return
optional int32 pagesize = 6;
}

// Response message for GetGroupContributions
message GetGroupContributionsResponse {
  // Group info
  Group group = 1;

  // Pagination information
  PageInfo pageinfo = 2;
}

// Group represents a GitLab group
message Group {
  // Group ID
  string id = 1;

  // Contributions within the specified date range
  ContributionConnection contributions = 2;
}

// ContributionConnection represents a paginated collection of contributions
message ContributionConnection {
  // List of contribution nodes
  repeated ContributionNode nodes = 1;

  // Pagination information
  PageInfo pageinfo = 2;
}

// ContributionNode represents a single user's contributions
message ContributionNode {
  // Number of repository pushes
  int32 repopushed = 1;

  // Number of merge requests created
  int32 mergerequestscreated = 2;

  // Number of merge requests merged
  int32 mergerequestsmerged = 3;

  // Number of merge requests closed
  int32 mergerequestsclosed = 4;
}

```

```

// Number of merge requests approved
int32 mergerequestsapproved = 5;

// Number of issues created
int32 issuescreated = 6;

// Number of issues closed
int32 issuesclosed = 7;

// Total number of events
int32 totalevents = 8;

// User information
User user = 9;
}

// User represents a GitLab user
message User {
  // User ID
  string id = 1;

  // User's name
  string name = 2;

  // URL to user's profile
  string weburl = 3;
}

// PageInfo provides pagination metadata
message PageInfo {
  // Token that can be used to fetch the next page
  // When empty or missing, there are no more pages after this one
  optional string nexttoken = 1;

  // Token that can be used to fetch the previous page
  // When empty or missing, there are no pages before this one
  optional string previoustoken = 2;
}

```

gRPC request

ruby

Step 1: Add required gems to your Gemfile

Gemfile

```
gem 'grpc'
```

```
gem 'google-protobuf'
```

Step 2: Generate Ruby code from your .proto file using protoc

You'd run this command in your terminal:

```
protoc --rubyout=lib/protos --grpcrubyout=lib/protos -I protos protos/groupcontributions.proto
```

Step 3: Create a client wrapper class in app/services/groupcontributionsclient.rb

```
module Services
  class GroupContributionsClient
    def initialize(host = 'localhost:50051', bearertoken)
      Use TLS/SSL for secure connection
      sslcreds = GRPC::Core::ChannelCredentials.new
      authcreds = bearertokencredentials(bearertoken)
      credentials = sslcreds.compose(authcreds)

      @stub = Gitlab::Group::Contributions::V1::GroupContributionsService::Stub.new(
        host,
        credentials
      )
    end

    def getgroupcontributions(fullpath, startdate, enddate, nexttoken = nil, previoustoken =
    nil, pagesize = 50)
      request = Gitlab::Group::Contributions::V1::GetGroupContributionsRequest.new(
        fullpath: fullpath,
        startdate: startdate,
        enddate: enddate,
        nexttoken: nexttoken,
        previoustoken: previoustoken,
        pagesize: pagesize
      )

      @stub.getgroupcontributions(request)
    end

    private

    def bearertokencredentials(token)
      procmetadatagenerator = proc { { 'authorization' => "Bearer {token}" } }
      GRPC::Core::CallCredentials.new(procmetadatagenerator)
    end
  end
end
```

Step 4: Use the client in your controller

```
class ContributionsController < ApplicationController
  def index
    client = Services::GroupContributionsClient.new(
      ENV['CONTRIBUTIONSGRPCHOST'],
      ENV['CONTRIBUTIONSAPITOKEN']
    )
```

Determine which token to use based on navigation direction

```
nexttoken = params[:nexttoken]
```

```
previoustoken = params[:previoustoken]
```

```
@response = client.getgroupcontributions(  
  params[:grouppath],  
  params[:startdate],  
  params[:enddate],  
  nexttoken,  
  previoustoken,  
  params[:pagesize] || 50  
)
```

Transform gRPC response to a format suitable for your views

```
@contributions = @response.group.contributions.nodes.map do |node|  
  {  
    user: {  
      id: node.user.id,  
      name: node.user.name,  
      url: node.user.weburl  
    },  
    metrics: {  
      repopushes: node.repopushed,  
      mrcreated: node.mergerequestscreated,  
      mrmerged: node.mergerequestsmerged,  
      mrclosed: node.mergerequestsclosed,  
      mrapproved: node.mergerequestsapproved,  
      issuescreated: node.issuescreated,  
      issuesclosed: node.issuesclosed,  
      totalevents: node.totalevents  
    }  
  }  
end
```

```
@pagination = {  
  nexttoken: @response.pageinfo.nexttoken,  
  previoustoken: @response.pageinfo.previoustoken  
}
```

```
respondto do |format|  
  format.html  
  format.json { render json: { contributions: @contributions, pagination: @pagination } }  
end  
end  
end
```

gRPC response

```
go  
// SERVER IMPLEMENTATION  
package main
```

```

import (
    "context"
    "log"
    "net"

    pb "gitlab.com/api/group/contributions/v1"
    "google.golang.org/grpc"
)

type groupContributionsServer struct {
    pb.UnimplementedGroupContributionsServiceServer
}

func (s groupContributionsServer) GetGroupContributions(ctx context.Context, req
pb.GetGroupContributionsRequest) (pb.GetGroupContributionsResponse, error) {
    // In a real implementation, you would:
    // 1. Parse dates
    // 2. Fetch data from database
    // 3. Format the response

    // Mock implementation for example
    return &pb.GetGroupContributionsResponse{
        Group: &pb.Group{
            Id: "gid://gitlab/Group/9970",
            Contributions: &pb.ContributionConnection{
                Nodes: []pb.ContributionNode{
                    {
                        RepoPushed:      5,
                        MergeRequestsCreated: 3,
                        MergeRequestsMerged: 2,
                        MergeRequestsClosed: 1,
                        MergeRequestsApproved: 4,
                        IssuesCreated:      2,
                        IssuesClosed:        1,
                        TotalEvents:        18,
                        User: &pb.User{
                            Id: "gid://gitlab/User/123456",
                            Name: "Jane Developer",
                            WebUrl: "https://gitlab.com/janedevloper",
                        },
                    },
                    // Additional nodes would be added here
                },
            },
            PageInfo: &pb.PageInfo{
                NextToken: "nexttokenxyz",
                PreviousToken: "", // Empty because this is the first page
            },
        },
        PageInfo: &pb.PageInfo{

```

```

    NextToken: "nexttokenxyz",
    PreviousToken: "", // Empty because this is the first page
  }, nil
}

func main() {
  lis, err := net.Listen("tcp", ":50051")
  if err != nil {
    log.Fatalf("failed to listen: %v", err)
  }

  grpcServer := grpc.NewServer()
  pb.RegisterGroupContributionsServiceServer(grpcServer, &groupContributionsServer{})

  log.Println("Starting gRPC server on port 50051...")
  if err := grpcServer.Serve(lis); err != nil {
    log.Fatalf("failed to serve: %v", err)
  }
}

```

// CLIENT IMPLEMENTATION

package main

```

import (
  "context"
  "log"
  "time"

  pb "gitlab.com/api/group/contributions/v1"
  "google.golang.org/grpc"
  "google.golang.org/grpc/credentials/insecure"
)

```

```

func main() {
  conn, err := grpc.Dial("localhost:50051",
    grpc.WithTransportCredentials(insecure.NewCredentials()))
  if err != nil {
    log.Fatalf("did not connect: %v", err)
  }
  defer conn.Close()

  client := pb.NewGroupContributionsServiceClient(conn)

  ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
  defer cancel()

  // Example of a first page request
  resp, err := client.GetGroupContributions(ctx, &pb.GetGroupContributionsRequest{
    FullPath: "gitlab-org",
  })
}

```

```

    StartDate: "2025-02-06",
    EndDate: "2025-02-13",
    NextToken: "", // Empty for initial request
    PreviousToken: "", // Empty for initial request
    PageSize: 50
})

if err != nil {
    log.Fatalf("could not get group contributions: %v", err)
}

// Process the response, e.g.:
log.Printf("Group ID: %s", resp.Group.Id)
log.Printf("Number of contributors: %d", len(resp.Group.Contributions.Nodes))

// Access pagination info
log.Printf("Next page token: %s", resp.PageInfo.NextToken)
log.Printf("Previous page token: %s", resp.PageInfo.PreviousToken)
log.Printf("Has next page: %v", resp.PageInfo.NextToken != "")
log.Printf("Has previous page: %v", resp.PageInfo.PreviousToken != "")

// Store tokens for navigation
nextPageToken := resp.PageInfo.NextToken

// Example of requesting the next page (if available)
if nextPageToken != "" {
    nextPageResp, err := client.GetGroupContributions(ctx, &pb.GetGroupContributionsRequest{
        FullPath: "gitlab-org",
        StartDate: "2025-02-06",
        EndDate: "2025-02-13",
        NextToken: nextPageToken, // Use next token from previous response
        PreviousToken: "", // Not needed when navigating forward
        PageSize: 50
    })
    if err != nil {
        log.Fatalf("could not get next page: %v", err)
    }

    // Process next page...
    log.Printf("Next page - contributors: %d", len(nextPageResp.Group.Contributions.Nodes))
}

// Access individual contributions
for i, node := range resp.Group.Contributions.Nodes {
    log.Printf("Contributor %d: %s", i+1, node.User.Name)
    log.Printf("Total group events: %d", node.TotalEvents)
    // And so on...
}
}

```

Example protobuf parameters

Group Contributions

- `datetimeBefore`
 - Only accepts UTC, conversion to and from user's timezone should be done by the API caller
 - Formatted to ISO-8601
- `datetimeAfter`
 - Only accepts UTC, conversion to and from user's timezone should be done by the API caller
 - Formatted to ISO-8601
- `fullPath`
 - Path to the GitLab group/namespace/project

Response codes

The following response codes must be returned by the Protobuf API:

- 200 OK - On success response
- 400 INVALIDARGUMENT - When a query is malformed
- 401 UNAUTHENTICATED - When a user is not authenticated
- 403 PERMISSIONDENIED - When a user is authenticated but doesn't have permission to query a particular resource
- 404 NOTFOUND - When a request is sent to an unknown endpoint
- 429 RESOURCEEXHAUSTED - When a request is trying to retrieve more data than the API can handle (querying all the data we have on every issue since time immemorial, for instance), or when rate limits are exceeded
- 500 INTERNAL - When the server fails unexpectedly

Response bodies should not expose the infrastructure or internal schematics of the DIP. All responses should be sent to our Metrics and Logs. Any error responses returned to the client must have user-friendly responses explaining the cause of the error without giving internal details.

Authentication, authorization, and security

Requests made to the DIP must pass through the Monolith. The DIP will not be accessible through direct endpoints.

The Monolith will authenticate and authorize all requests for data as per the normal Monolith processes.

Since the DIP is used only for internal communication, and all requests are pre-authorized by the Monolith, the DIP will use a shared authentication token to communicate with the Monolith via gRPC, in the same way as Gitaly already does. We must have a way to rotate this shared authentication token. See the Gitaly rotation runbook for how we do this for .com.

All requests to and from DIP, including internal network requests, should be encrypted using TLS or other appropriate encryption methods. Any data stored within the data flow of the querying API must be encrypted at rest (caches for example).

Performance

To keep query times down, as well as to optimize our data retrieval in general, we will likely heavily leverage ClickHouse's pre-aggregated Materialized Views (MVs) wherever possible. This approach moves complex joining of data away from when data is SELECTed to when it is INSERTed. ClickHouse tends to perform better with larger tables, compared to JOINS.

Due to the asynchronous nature of ClickHouse data merging, queries will likely need to use GROUP BY or FINAL to ensure the correctness of queried data.

Each resource must document in our docs any limits that must be applied to avoid excessive query times. The resource must validate that these limits are not exceeded by any given query. If a query attempts to exceed these limits, a 429 RESOURCEEXHAUSTED error must be returned with the reason why. For instance, a resource can only support queries of up to 30 days, and a query requests 60 days.

Rate limiting and IP blocking

For the GraphQL and REST APIs, as these will run through the existing GitLab framework, we should leverage the existing rate limiting and IP blocking capabilities.

Rate limiting headers (RateLimit-) must be returned on API requests for the frontend to manage/inform the user if queries are being slowed down, or if the customer is automating API requests. Requests exceeding the defined rate limit should return a 429 RESOURCEEXHAUSTED status code.

For the gRPC API, rather than defining rate limits, it is recommended to use a concurrency limit instead. Ideally, this should be adaptive, automatically backing off requests when a specific data source is getting overloaded. Each limit should be customisable per data source, as each one will have different limitations.

Metrics and logs

We must make sure that the Protobuf API is integrated with our existing observability infrastructure. All logs should be sent to Kibana, whilst frontend GitLab usage errors and performance should be sent to Sentry.

Logs must not store unnecessary personally identifiable information, secrets, or keys. All new logging calls must be checked to make sure we're not leaking information into our logs that we shouldn't.

ClickHouse queries must be logged in a redacted format, replacing placeholders with a ?. Query logs must have a hash of the query for easy comparison and finding of similar queries.

Errors must contribute to our Error Budget, so we can monitor improvements over time.

We must investigate how we can closely integrate with our Internal Analytics to track usage. We will add metrics to the frontend GitLab usage, but integration with the DIP needs investigation to see if it is feasible, or if we need to design a secondary API for sending events (Service Ping or the Events Tracking API?).

Metrics must be sent to Prometheus for us to track (not exhaustive):

- API response times
- API uptimes
- Resource usages
- Slow queries
- Expensive queries

Pagination

As outlined above, we must use keyset-based pagination where possible. For aggregated queries we will need to return the full query, but we should be able to mitigate some of the performance concerns by building pre-defined MVs for aggregated queries and using our metrics to monitor this over time.

Docs

We must create a quick start guide explaining how to connect and use the Protobuf API. This documentation should be aimed at users with limited knowledge of the DIP. The documentation must clearly outline the flow of data and which teams are responsible for each part of the flow.

In conjunction with the automation of the schema, we should publish the schema details, with all restrictions and parameters, to docs.gitlab.com.

We must create a guide on how to create MV's for a given resource, and how to use these within the DIP to improve query times and performance.

Considerations for .com/dedicated/cells/self-managed

This API's queries will not span multiple cells. Each query must be scoped to an instance (admin users of self-managed instances only), organization, namespace, group or project.

One of the core requirements for the DIP is for it to be deployed as part of the GitLab instance, similar to Gitaly. Therefore, all our customers should have access to the DIP in full.

SECTION: engineering/architecture/design-documents/dependency_scanning_analyzer/_index.md

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

<!--

Don't add a h1 headline. It'll be added automatically from the title front matter attribute.
For long pages, consider creating a table of contents.

-->

Summary

The dependency scanning feature is powered by a set of analyzers - gemnasium, gemnasium-maven, and gemnasium-python. Associated with CI templates, these analyzers have the responsibility of detecting supported projects, building the dependency graph or list when needed, parsing the detected dependencies, and finally, producing a security report with detected vulnerabilities alongside a CycloneDX SBOM that contains the dependencies. This approach has worked well, but over time it's become evident that the actions required to build a project's dependency graph exports come with a lot of complexity. This complexity negatively impacts the maintenance and creation of features, and the user experience of setting up and maintaining the dependency scanning analyzer.

Motivation

The high cost associated with building the dependency graphs/list exports motivates us to rethink how we can structure the dependency scanning feature. Instead of building the project dependency graphs or lists for customers, we can delegate this responsibility to a job that runs before the analyzer does. A build stage is a very common part of the development cycle, and generating the dependency artifacts during this stage is a lot simpler than mapping existing build system configuration values to the ones used by the gemnasium set of analyzers.

In addition, build jobs can take a considerably large amount of time, so removing the build process from dependency scanning reduces user's CI minute usage, and further tightens the development feedback loop. Further yet, building a project twice presents the possibility that the analyzer may build something that does not match what is deployed. This mismatch can lead to false positives and negatives, both of which skew a project's security status signal.

Goals

- Customers remove the need to set up a secondary build process. Historically, Python and Java have required a build process.
- Reduced bug maintenance costs. A large amount of our issues surface from edge cases that are already handled by a customer in a previous build step, but were not accounted for by the analyzer's build implementation. These issues increase the code complexity, and cut into scheduled additions and improvements.
- Offline support by default.
- Reduced security maintenance costs. Building projects means that the analyzer images need to ship with pre-installed versions of supported build systems, for example Gradle and Maven, and runtimes like Java or Python.
- Removal of historical limitations like single project analysis for Java and Python monorepos.

Non-Goals

- Supporting 3rd party SBOM generators. We can still support this in a future iteration.

Proposal

Create a new analyzer that focus on supporting only dependency graph exports.

Document how to generate the exports with example projects, and provide a dependency scanning CI/CD component that scans the generated artifacts.

Because of the change to SBOM-based scanning in epic 8026, do not port over the vulnerability matching done by the Gemnasium analyzers, as this functionality is already planned for deprecation.

The new analyzer should be based on a scratch image to reduce the attack surface introduced by container dependencies.

Pros

- Simplified integration tests. No need to test against various permutations of package managers, runtime, and compiler versions.
- We should always have zero container-scanning vulnerabilities. This translates to a reduced workload on the engineers going through reaction rotation.
- Smaller image sizes. Fast CI job start-up, reduced network traffic.
- Simplified FIPS-compliance as the library does not use crypto libraries.
- Improved community contribution experience due to simplified permissions for development pipeline execution.

Cons

- Additional documentation required with examples and guides on getting started with a dependency scanning for certain package managers.
- Requires the establishment of new graph export naming standards.
- Users need to configure their build jobs as instructions. It doesn't work out of the box.

Design and implementation details

At a high level, the new dependency scanning feature will operate as follows.

mermaid

sequenceDiagram

autonumber

actor user

participant build job

participant analyzer

participant sbom ingestion service

participant database

user->>build job: triggers pipeline on default branch

build job->>analyzer: passes all dependency graph exports generated as artifacts

analyzer->>sbom ingestion service: converts dependency graph exports to CycloneDX SBOMs

sbom ingestion service->>database: stores occurrences of sbom components

Build job(s)

It's important to note that we cannot expect for a dependency graph export to be checked into a project's repository. This is likely to happen in cases where the dependency graph export does not also function as a lock file like in the cases of pipdeptree and pipenv graph dependency graph exports. In such cases, we will expect the build job to generate the dependency graph exports, and for the job to store these as job artifacts.

We'll use the following naming conventions to establish a contract with users on what file's we'll detect in cases where the dependency graph export does not function as a lock file, and thus does not have a canonical name.

Pattern	Description
/go.graph	Generated via go mod graph > go.graph
/pipenv.graph.json	Generated via pipenv graph --tree-json > pipenv

It's required for the build job to run in a stage that precedes the one in which the dependency scanning analyzer runs. This is true by default, since the analyzer runs in the test stage which runs after the build stage.

Analyzer

Once the build jobs complete, and the artifacts are stored, they will be passed on to proceeding jobs unless specifically asked not to do so. The analyzer takes advantage of this and expects that users have configured the build jobs to pass on the artifacts using the documented naming patterns. It will then search the entire target directory, by default this is the project's repository, detect all supported files, parse them, and convert them into a CycloneDX SBOM that can be utilized by the services running in the GitLab monolith.

Pros

- No preinstalled compilers, runtimes or system dependencies required.
- Small attack surface.
- Runs offline by default.

Cons

- Graph export documentation varies in quality. Some package managers like npm document each version of the lock file, while others like pnpm do not.
- Java and Python projects require additional configuration since they do not capture graph information in their lock files by default.

Alternative Solutions

Require lock file, add graph information to it

One alternative solution to dependency graph exports is to make every supported

lock file a dependency graph export by default. In this scenario, we would